

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Application Development and Jira

PRACTICAL 7

Roll No.: T005	Name: Ritesh Chaubey
Class: TYIT	Batch: 1
Date of Assignment: 05-08-2026	Date/Time of Submission: 05-08-2026

AIM :- Machine Learning

2) Using a suitable regression dataset, implement Linear Regression using scikit-Learn.

CODE :-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
data = pd.read_csv("student_marks.csv")
x = data[['Hours']]
y = data['Marks']
x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.3,
    random_state=1
)
model = LinearRegression()
model.fit(x_train, y_train)
print("Training Accuracy:",
      round(model.score(x_train, y_train) * 100, 2))
print("Testing Accuracy:",
      round(model.score(x_test, y_test) * 100, 2))

# Predict marks for 11 hours of study
new_data = pd.DataFrame([[11]], columns=['Hours'])
pred = model.predict(new_data)
print("Predicted Marks for 11 hours:", round(pred[0], 2))
```

Output:-

```
Training Accuracy: 99.64
Testing Accuracy: 98.71
Predicted Marks for 11 hours: 93.57
Ritesh Chaubey T005
```

3) Using the MNIST dataset (digit vs not digit), implement a binary classifier.

CODE :-

```
Import pandas as pd
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
data = pd.read_csv("mnist_test.csv")
digits = load_digits()
X = digits.data
y = digits.target
y = (y == 5)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
print("Training Accuracy:", round(model.score(X_train, y_train) * 100, 2), "%")
print("Testing Accuracy:", round(model.score(X_test, y_test) * 100, 2), "%")
prediction = model.predict([X_test[0]])
if prediction[0]:
    print("Digit is 5")
else:
    print("Digit is not 5")
print("Ritesh Chaubey T005")
```

Output:-

```
Training Accuracy: 100.0 %
Testing Accuracy: 98.89 %
Digit is not 5
Ritesh Chaubey T005
```

1. California Housing Dataset

Problem Statement

A real estate company wants to predict the median house value based on housing features.

Features

- Median Income
- House Age
- Average Rooms
- Average Bedrooms
- Population
- Average Occupancy
- Latitude
- Longitude

Target

Median House Value

Tasks

1. Load the dataset using Pandas.
2. Display the first five records.
3. Check for missing values.
4. Select one suitable feature (e.g., Median Income) as the independent variable.
5. Split the dataset into training and testing sets.
6. Train a Linear Regression model.
7. Predict the house prices.
8. Display:
 - o Coefficient
 - o Intercept
 - o R2 Score
9. Plot the regression line.
10. Interpret the results.

Code:-

```
# Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# 1. Load the dataset
df = pd.read_csv("housing.csv")

# 2. Display first five records
print("First Five Records:")
print(df.head())
```

```
# 3. Check for missing values
print("\nMissing Values:")
print(df.isnull().sum())
# Remove missing values if any
df = df.dropna()
# 4. Select one feature (Median Income) as Independent Variable
X = df[['median_income']]
# Target Variable
y = df['median_house_value']
# 5. Split dataset into Training and Testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# 6. Train Linear Regression Model
model = LinearRegression()
model.fit(X_train, y_train)
# 7. Predict House Prices
y_pred = model.predict(X_test)
# 8. Display Coefficient, Intercept and R2 Score
print("\nCoefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
print("R2 Score:", r2_score(y_test, y_pred))
# 9. Plot Regression Line
plt.figure(figsize=(8,6))
plt.scatter(X_test, y_test, color='blue', label='Actual Data')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Regression Line')
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("Linear Regression - California Housing Dataset")
plt.legend()
plt.show()
# 10. Display Actual vs Predicted Values
result = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred
})
print("\nActual vs Predicted Values:")
print(result.head(10))
```

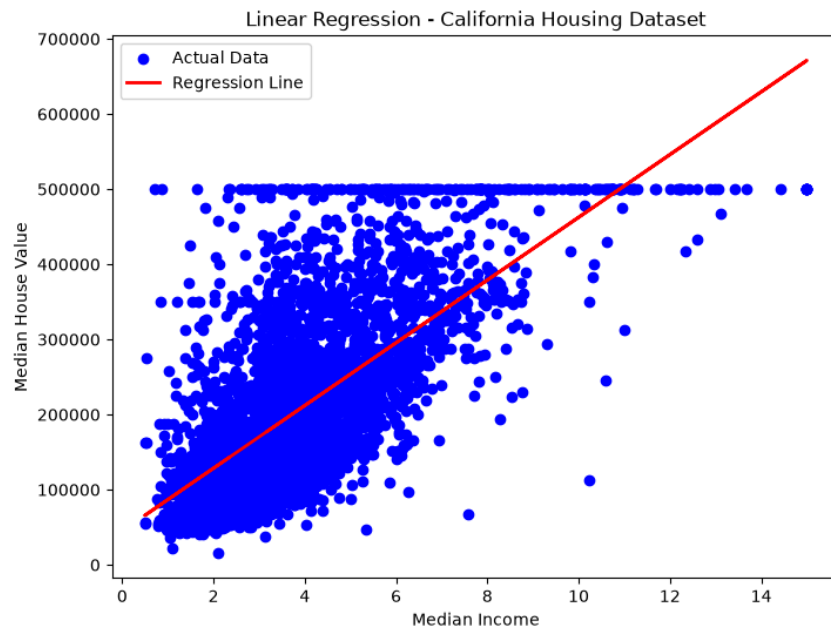
Output:-

```
First Five Records:
  longitude  latitude  ...  median_house_value  ocean_proximity
0   -122.23    37.88  ...           452600         NEAR BAY
1   -122.22    37.86  ...           358500         NEAR BAY
2   -122.24    37.85  ...           352100         NEAR BAY
3   -122.25    37.85  ...           341300         NEAR BAY
4   -122.25    37.85  ...           342200         NEAR BAY

[5 rows x 10 columns]

Missing Values:
longitude           0
latitude           0
housing_median_age  0
total_rooms         0
total_bedrooms     207
population         0
households         0
median_income       0
median_house_value  0
ocean_proximity     0
dtype: int64

Coefficient: 41751.95797204511
Intercept: 45035.22817538006
R2 Score: 0.47196228574894983
```



Actual vs Predicted Values:

	Actual	Predicted
0	245800	177965.111967
1	137900	232380.438792
2	218200	179631.015090
3	220800	165781.890631
4	170500	216285.058994
5	75000	130104.842543
6	269400	264349.913011
7	228900	249227.353833
8	500001	363306.228600
9	80800	133758.138866

Questions on Part 3

Q.1 Modify the program to detect Digit 7 vs Not Digit 7 instead of Digit 5 vs Not Digit 5.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
data = pd.read_csv("mnist_test.csv")
digits = load_digits()
X = digits.data
y = digits.target
y = (y == 7)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
print("Training Accuracy:", round(model.score(X_train, y_train) * 100, 2), "%")
print("Testing Accuracy:", round(model.score(X_test, y_test) * 100, 2), "%")
prediction = model.predict([X_test[0]])
if prediction[0]:
    print("Digit is 7")
else:
    print("Digit is not 7")
print("Ritesh Chaubey T005")
```

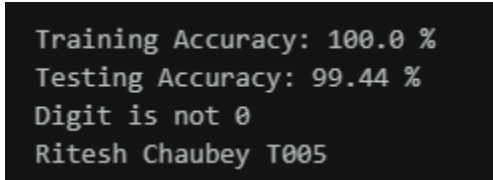
```
Training Accuracy: 100.0 %
Testing Accuracy: 99.07 %
Digit is not 7
Ritesh Chaubey T005
```

Q.2 Modify the program to detect Digit 0 vs Not Digit 0.

Code:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
data = pd.read_csv("mnist_test.csv")
digits = load_digits()
X = digits.data
y = digits.target
y = (y == 0)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
print("Training Accuracy:", round(model.score(X_train, y_train) * 100, 2), "%")
print("Testing Accuracy:", round(model.score(X_test, y_test) * 100, 2), "%")
prediction = model.predict([X_test[0]])
if prediction[0]:
    print("Digit is 0")
else:
    print("Digit is not 0")
print("Ritesh Chaubey T005")
```

Output:-



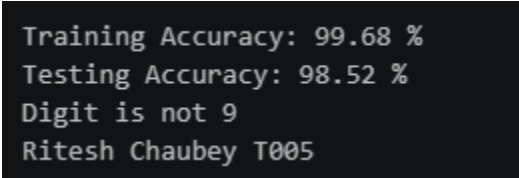
```
Training Accuracy: 100.0 %
Testing Accuracy: 99.44 %
Digit is not 0
Ritesh Chaubey T005
```

Q.3 Modify the program to detect Digit 9 vs Not Digit 9.

Code:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
data = pd.read_csv("mnist_test.csv")
digits = load_digits()
X = digits.data
y = digits.target
y = (y == 9)
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.3, random_state=1
)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
print("Training Accuracy:", round(model.score(X_train, y_train) * 100, 2), "%")
print("Testing Accuracy:", round(model.score(X_test, y_test) * 100, 2), "%")
prediction = model.predict([X_test[0]])
if prediction[0]:
    print("Digit is 9")
else:
    print("Digit is not 9")
print("Ritesh Chaubey T005")
Output:-
```



```
Training Accuracy: 99.68 %
Testing Accuracy: 98.52 %
Digit is not 9
Ritesh Chaubey T005
```

Q.4 Modify the program to display the first 20 predictions instead of only one prediction.

Code:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_digits
data = pd.read_csv("mnist_test.csv")
digits = load_digits()
X = digits.data
y = digits.target
y = (y == 7)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
print("Training Accuracy:", round(model.score(X_train, y_train) * 100, 2), "%")
print("Testing Accuracy:", round(model.score(X_test, y_test) * 100, 2), "%")
# Display first 20 predictions
predictions = model.predict(X_test[:20])
print("\nFirst 20 Predictions:")
for i in range(20):
```

```

print("Sample", i + 1)
if predictions[i]:
    print("Predicted: Digit is 7")
else:
    print("Predicted: Digit is not 7")
if y_test[i]:
    print("Actual: Digit is 7")
else:
    print("Actual: Digit is not 7")
print("-----")
print("Ritesh Chaubey T005")
Output:-

```

```

Training Accuracy: 100.0 %
Testing Accuracy: 99.07 %

```

```

First 20 Predictions:

```

```

Sample 1
Predicted: Digit is not 7
Actual: Digit is not 7
-----

```

```

Sample 2
Predicted: Digit is not 7
Actual: Digit is not 7
-----

```

```

Sample 3
Predicted: Digit is not 7
Actual: Digit is not 7
-----

```

```

Sample 4
Predicted: Digit is 7
Actual: Digit is 7
-----

```

```

Sample 5
Predicted: Digit is not 7
Actual: Digit is not 7
-----

```

```

Sample 6
Predicted: Digit is not 7
Actual: Digit is not 7
-----

```

```

-----
Sample 7
Predicted: Digit is not 7
Actual: Digit is not 7
-----

```

```

Sample 8
Predicted: Digit is not 7
Actual: Digit is not 7
-----

```

```

Sample 9
Predicted: Digit is not 7
Actual: Digit is not 7
-----

```

```

Sample 10
Predicted: Digit is not 7
Actual: Digit is not 7
-----

```

```

Sample 11
Predicted: Digit is not 7
Actual: Digit is not 7
-----

```

```

Sample 12
Predicted: Digit is not 7
Actual: Digit is not 7
-----

```

```

Sample 13
Predicted: Digit is 7
Actual: Digit is 7
-----

```

```

Sample 14
Predicted: Digit is not 7
Actual: Digit is not 7
-----

```

```

Sample 15
Predicted: Digit is not 7
Actual: Digit is not 7
-----

```

```

Sample 16
Predicted: Digit is not 7
Actual: Digit is not 7
-----

```

```

Sample 17
Predicted: Digit is not 7
Actual: Digit is not 7
-----

```

```

Sample 18
Predicted: Digit is not 7
Actual: Digit is not 7
-----

```

```

Sample 19
Predicted: Digit is 7
Actual: Digit is 7
-----

```

```

Sample 20
Predicted: Digit is not 7
Actual: Digit is not 7
-----

```

```

Ritesh Chaubey T005

```